The Normalization Approach to Database Design

Normalization is a systematic process used in database design to organize data into well-structured tables and reduce data redundancy.

Unlike the ER/EER modeling approach, which begins by identifying entities and relationships, the normalization approach starts with a collection of data items or an existing table and then decomposes it into smaller, more manageable relations.

Objectives of Normalization

The primary goals of normalization are to:

- Eliminate redundant data
- Reduce data anomalies
- Improve data consistency
- Simplify database maintenance
- Ensure efficient data storage
- Preserve data integrity

Data Anomalies

Poorly designed tables often suffer from three types of anomalies.

1. **Insertion Anomaly**

An insertion anomaly occurs when data cannot be added without adding unrelated information.

Example:

A new course cannot be added until at least one student enrolls in it.

For example, suppose a table stores:

Student ID	Student Name	Course
101	Alice	Mathematics
102	Bob	Physics

If you want to add a new course, **Chemistry**, you can't record it until a student enrolls—unless you leave the student fields empty or enter fake data.

That's an insertion anomaly. Separating **Students**, **Courses**, and **Enrollments** into their own tables solves it.

2. **Update Anomaly**

An update anomaly occurs when the same information appears in multiple records and must be updated in several places.

Example:

If an instructor's room changes, multiple rows must be updated.

For example:

Student	Course	Instructor's Office
Alice	Mathematics	Room 201
Bob	Mathematics	Room 201

If the instructor moves to **Room 305**, both rows need updating. If only Alice's row changes, the table shows two different offices for the same instructor.

Separating instructor information into its own table solves this: the office is stored and updated once.

3. **Deletion Anomaly**

A deletion anomaly occurs when deleting one piece of information unintentionally removes other important data.

Example:

Deleting the last student enrolled in a course also removes information about the course and instructor information itself.

For example:

Student	Course	Instructor
Alice	Mathematics	Dr. Smith
Bob	Physics	Dr. Jones

If Bob withdraws and you delete his row, you also lose the information that **Dr. Jones teaches Physics**, because Bob was the only student listed for that course.

Separating course information from student enrollments prevents this: deleting an enrollment keeps the course and instructor information.

Database Normalization

Database normalization is the process of organizing data into related tables to reduce unnecessary duplication and improve data consistency. It helps prevent insertion, update, and deletion anomalies, which occur when adding, changing, or removing data causes unintended problems. Normalization follows rules called normal forms:

- First Normal Form (1NF) ensures each cell contains a single value;
- Second Normal Form (2NF) removes dependencies on only part of a candidate key; and
- Third Normal Form (3NF) removes transitive dependencies of non-key attributes on keys.

First Normal Form (1NF)

First Normal Form (1NF) is a basic rule for organizing a relational database. A table is in 1NF when each cell contains a single value and there are no repeating groups of columns. Each row should also be uniquely identifiable, usually through a primary key.

Not in 1NF

StudentID	**Name**	**PhoneNumbers**
1001	John	555-1111, 555-2222

The PhoneNumbers field contains multiple values.

1NF Solution

Student

StudentID	Name
1001	John

StudentPhone

StudentID	PhoneNumber
1001	555-1111
1001	555-2222

Not in 1NF

A table with **duplicate rows** has rows that cannot be uniquely identified. For example:

Student Name	**Course**
Alice Smith	Mathematics
Alice Smith	Mathematics
Bob Jones	Physics

The two Alice Smith rows contain the same values (two student with the same first and last names), so there is no way to identify one separately from the other.

1NF Solution

Adding a **Student ID** can distinguish different students:

Student ID	Student Name	Course
101	Alice Smith	Mathematics
102	Alice Smith	Mathematics
103	Bob Jones	Physics

Second Normal Form (2NF)

Second Normal Form (2NF) helps reduce duplicated information. A table is in 2NF when:

1. It is already in **First Normal Form (1NF)**.
2. Every non-key attribute depends on the **whole of every candidate key**, rather than just part of one.

Example

Consider this enrollment table:

Student ID	Course ID	Student Name	Grade
101	C01	Alice	A
101	C02	Alice	B
102	C01	Bob	B

The primary key is **Student ID + Course ID**, which uniquely identifies each enrollment.

- **Grade** depends on both the student and the course.
- **Student Name** depends only on Student ID.

Because Student Name depends on only part of the key, this is a **partial dependency**, and the table is **not in 2NF**. Alice's name is repeated for every course she takes, which can cause update anomalies.

To achieve 2NF, split the table:

Students

Student ID	Student Name
101	Alice
102	Bob

Enrollments

Student ID	Course ID	Grade

101	C01	A
101	C02	B
102	C01	B

Student names are now stored once, and each non-key attribute depends on the whole key of its table.

Third Normal Form (3NF)

Third Normal Form (3NF) reduces duplication by removing **transitive dependencies**. In a typical table, this means non-key attributes should depend on the key, rather than on another non-key attribute.

A table is in 3NF when:

1. It is already in **Second Normal Form (2NF)**.
2. It has no transitive dependencies of non-key attributes on a candidate key.

For example:

Student ID	Student Name	Department ID	Department Name
101	Alice	D01	Computing
102	Bob	D01	Computing
103	Carol	D02	Business

The primary key is **Student ID**. However, Department Name depends on Department ID:

Student ID → Department ID → Department Name

This is a transitive dependency, so the table is **not in 3NF**. If Computing is renamed, every student row for that department must be updated, creating the possibility of an update anomaly.

To achieve 3NF, split the table:

Students

Student ID	Student Name	Department ID
101	Alice	D01
102	Bob	D01
103	Carol	D02

Departments

Department ID	Department Name
D01	Computing
D02	Business

Each department's name is now stored once. Department ID links the two tables, so you can still find each student's department name.

Summary

Third Normal Form (3NF) is usually enough for many practical databases, because it addresses most common causes of duplication and insertion, update, and deletion anomalies. However, it does not eliminate every possible anomaly.

More complex relationships may require higher normal forms, such as **Boyce–Codd Normal Form (BCNF), Fourth Normal Form (4NF), or Fifth Normal Form (5NF)**.